

VLLM专题（二十）—多模态输入



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续



您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

本页面将教你如何在 vLLM 中向多模态模型传递多模态输入。

1. 离线推理

要输入多模态数据，请按照以下 `vllm.inputs.PromptType` 中的格式：

- **prompt**: 提示词（prompt）应遵循 HuggingFace 文档中记录的格式。
- **multi_modal_data**: 这是一个字典，遵循 `vllm.multimodal.inputs.MultiModalDataDict` 中定义的格式。

1.1 图像输入

你可以将单张图像传递给多模态字典中的 `'image'` 字段，如下例所示：

```

llm = LLM(model="llava-hf/llava-1.5-7b-hf")

# Refer to the HuggingFace repo for the correct format to use
prompt = "USER: <image>\nWhat is the content of this image?\nASSISTANT:"

# Load the image using PIL.Image
image = PIL.Image.open(...)

# Single prompt inference
outputs = llm.generate({

    "prompt": prompt,
    "multi_modal_data": {
        "image": image},
})

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

# Batch inference
image_1 = PIL.Image.open(...)
image_2 = PIL.Image.open(...)
outputs = llm.generate(
    [
        {

            "prompt": "USER: <image>\nWhat is the content of this image?\nASSISTANT:",
            "multi_modal_data": {
                "image": image_1},
        },
        {

            "prompt": "USER: <image>\nWhat's the color of this image?\nASSISTANT:",
            "multi_modal_data": {
                "image": image_2},
        }
    ]
)

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

```

完整示例：

你可以在如下 `vision_language.py` 中找到完整的代码示例。

```
# SPDX-License-Identifier: Apache-2.0
```

```
"""
```

This example shows how to use vLLM for running offline inference with the correct prompt format on vision language models for text generation.

For most models, the prompt format should follow corresponding examples on HuggingFace model repository.

```
"""
```

```
import os
```

```
import random
```

```
from huggingface_hub import snapshot_download
```

```
from transformers import AutoTokenizer
```

```
from vllm import LLM, SamplingParams
```

```
from vllm.assets.image import ImageAsset
```

```
from vllm.assets.video import VideoAsset
```

```
from vllm.lora.request import LoRARequest
```

```
from vllm.utils import FlexibleArgumentParser
```

```
# NOTE: The default `max_num_seqs` and `max_model_len` may result in OOM on
```

```
# Lower-end GPUs.
```

```
# Unless specified, these settings have been tested to work on a single L4.
```

```
# Aria
```

```
def run_aria(questions: list[str], modality: str):
```

```
    assert modality == "image"
```

```
    model_name = "rhymes-ai/Aria"
```

```
# NOTE: Need L40 (or equivalent) to avoid OOM
```

```
    llm = LLM(model=model_name,
```

```
              max_model_len=4096,
```

```
              max_num_seqs=2,
```

```
              dtype="bfloat16",
```

```
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
```

```
    prompts = [(f"<|im_start|>user\n<fim_prefix><|img|><fim_suffix>{  
                question}")
```

```
                "<|im_end|>\n<|im_start|>assistant\n")
```

```
                for question in questions]
```

```
    stop_token_ids = [93532, 93653, 944, 93421, 1019, 93653, 93519]
```

```
    return llm, prompts, stop_token_ids
```

```
# BLIP-2
```

```
def run_blip2(questions: list[str], modality: str):
```

```

def run_blip2(questions: list[str], modality: str):
    assert modality == "image"

    # BLIP-2 prompt format is inaccurate on HuggingFace model repository.
    # See https://huggingface.co/Salesforce/blip2-opt-2.7b/discussions/15#64ff02f3f8cf9e4
    # f5b038262 #noqa
    prompts = [f"Question: {
                question} Answer:" for question in questions]
    llm = LLM(model="Salesforce/blip2-opt-2.7b",
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Chameleon
def run_chameleon(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"{
                question}<image>" for question in questions]
    llm = LLM(model="facebook/chameleon-7b",
              max_model_len=4096,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Deepseek-VL2
def run_deepseek_vl2(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "deepseek-ai/deepseek-vl2-tiny"

    llm = LLM(model=model_name,
              max_model_len=4096,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
              hf_overrides={
                  "architectures": ["DeepseekVLV2ForCausalLM"]})

    prompts = [
        f"<|User|>: <image>\n{
            question}\n\n<|Assistant|>:"
        for question in questions
    ]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

```

# Florence2
def run_florence2(question: str, modality: str):
    assert modality == "image"

    llm = LLM(model="microsoft/Florence-2-large",
              tokenizer="facebook/bart-large",
              max_num_seqs=8,
              trust_remote_code=True,
              dtype="bfloat16",
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)

    prompt = "<MORE_DETAILED_CAPTION>"
    stop_token_ids = None
    return llm, prompt, stop_token_ids

# Fuyu
def run_fuyu(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"{
                question}\n" for question in questions]
    llm = LLM(model="adept/fuyu-8b",
              max_model_len=2048,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Gemma 3
def run_gemma3(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "google/gemma-3-4b-it"

    llm = LLM(
        model=model_name,
        max_model_len=2048,
        max_num_seqs=2,
        # Default is False; setting it to True is not supported in V1 yet
        mm_processor_kwargs={
            "do_pan_and_scan": True},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )
    prompts = [f"/<bos><start_of_turn>user\n"

```

```

        f"<start_of_image>{
        question}<end_of_turn>\n"
        "<start_of_turn>model\n") for question in questions]
stop_token_ids = None
return llm, prompts, stop_token_ids

# GLM-4v
def run_glm4v(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "THUDM/glm-4v-9b"

    llm = LLM(model=model_name,
               max_model_len=2048,
               max_num_seqs=2,
               trust_remote_code=True,
               enforce_eager=True,
               hf_overrides={
                   "architectures": ["GLM4VForCausalLM"]},
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)

    prompts = [
        f"<|user|>\n<|begin_of_image|><|endoftext|><|end_of_image|>\n"
        {
            question}<|assistant|>" for question in questions
    ]

    stop_token_ids = [151329, 151336, 151338]
    return llm, prompts, stop_token_ids

# H2OVL-Mississippi
def run_h2ovl(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "h2oai/h2ovl-mississippi-800m"

    llm = LLM(
        model=model_name,
        trust_remote_code=True,
        max_model_len=8192,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    tokenizer = AutoTokenizer.from_pretrained(model_name,
                                              trust_remote_code=True)

    messages = [[{

```

```

        'role': 'user',
        'content': f"<image>\n{
            question}"
    }] for question in questions]
    prompts = tokenizer.apply_chat_template(messages,
                                           tokenize=False,
                                           add_generation_prompt=True)

    # Stop tokens for H2OVL-Mississippi
    # https://huggingface.co/h2oai/h2ovl-mississippi-800m
    stop_token_ids = [tokenizer.eos_token_id]
    return llm, prompts, stop_token_ids

# Idefics3-8B-LLama3
def run_idefics3(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "HuggingFaceM4/Idefics3-8B-LLama3"

    llm = LLM(
        model=model_name,
        max_model_len=8192,
        max_num_seqs=2,
        enforce_eager=True,
        # if you are running out of memory, you can reduce the "longest_edge".
        # see: https://huggingface.co/HuggingFaceM4/Idefics3-8B-LLama3#model-optimization

        mm_processor_kwargs={

            "size": {

                "longest_edge": 3 * 364

            },

        },
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )
    prompts = [(
        f"<|begin_of_text|>User:<image>{
            question}<end_of_utterance>\nAssistant:"
    ) for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# InternVL
def run_internvl(questions: list[str], modality: str):

```

```

assert modality == "image"

model_name = "OpenGVLab/InternVL2-2B"

llm = LLM(
    model=model_name,
    trust_remote_code=True,
    max_model_len=4096,
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)

tokenizer = AutoTokenizer.from_pretrained(model_name,
                                         trust_remote_code=True)

messages = [[{
    'role': 'user',
    'content': f"<image>\n{
        question}"
}] for question in questions]
prompts = tokenizer.apply_chat_template(messages,
                                       tokenize=False,
                                       add_generation_prompt=True)

# Stop tokens for InternVL
# models variants may have different stop tokens
# please refer to the model card for the correct "stop words":
# https://huggingface.co/OpenGVLab/InternVL2-2B/blob/main/conversation.py
stop_tokens = ["<|endoftext|>", "<|im_start|>", "<|im_end|>", "<|end|>"]
stop_token_ids = [tokenizer.convert_tokens_to_ids(i) for i in stop_tokens]
return llm, prompts, stop_token_ids

# LLaVA-1.5
def run_llava(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [
        f"USER: <image>\n{
            question}\nASSISTANT:" for question in questions
    ]

    llm = LLM(model="llava-hf/llava-1.5-7b-hf",
              max_model_len=4096,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

```



```

# LLaVA-1.6/LLaVA-NeXT
def run_llava_next(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"[INST] <image>\n{
                question} [/INST]" for question in questions]
    llm = LLM(model="llava-hf/llava-v1.6-mistral-7b-hf",
              max_model_len=8192,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# LLaVA-NeXT-Video
# Currently only support for video input
def run_llava_next_video(questions: list[str], modality: str):
    assert modality == "video"

    prompts = [
        f"USER: <video>\n{
            question} ASSISTANT:" for question in questions
    ]
    llm = LLM(model="llava-hf/LLaVA-NeXT-Video-7B-hf",
              max_model_len=8192,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# LLaVA-OneVision
def run_llava_onevision(questions: list[str], modality: str):

    if modality == "video":
        prompts = [
            f"<|im_start|>user <video>\n{
                question}<|im_end|> \
            <|im_start|>assistant\n" for question in questions
        ]

    elif modality == "image":
        prompts = [
            f"<|im_start|>user <image>\n{
                question}<|im_end|> \
            <|im_start|>assistant\n" for question in questions
        ]

    llm = LLM(model="llava-hf/llava-onevision-qwen2-7b-ov-hf",

```

```

        max_model_len=16384,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Mantis
def run_mantis(questions: list[str], modality: str):
    assert modality == "image"

    llama3_template = '<|start_header_id|>user<|end_header_id|>\n\n{<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n\n' # noqa: E501
    prompts = [
        llama3_template.format(f"{
            question}\n<image>")
        for question in questions
    ]

    llm = LLM(
        model="TIGER-Lab/Mantis-8B-siglip-llama3",
        max_model_len=4096,
        hf_overrides={
            "architectures": ["MantisForConditionalGeneration"]},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )
    stop_token_ids = [128009]
    return llm, prompts, stop_token_ids

# MiniCPM-V
def run_minicpmv_base(questions: list[str], modality: str, model_name):
    assert modality in ["image", "video"]
    # If you want to use `MiniCPM-o-2_6` with audio inputs, check `audio_language.py` # noqa

    # 2.0
    # The official repo doesn't work yet, so we need to use a fork for now
    # For more details, please see: See: https://github.com/vllm-project/vllm/pull/4087#issuecomment-2250397630 # noqa
    # model_name = "HwvH/MiniCPM-V-2"

    # 2.5
    # model_name = "openbmb/MiniCPM-LLaMA3-V-2_5"

    # 2.6
    # model_name = "openbmb/MiniCPM-V-2_6"
    # o2.6

```

```

# modality supports
# 2.0: image
# 2.5: image
# 2.6: image, video
# o2.6: image, video, audio
# model_name = "openbmb/MiniCPM-o-2_6"
tokenizer = AutoTokenizer.from_pretrained(model_name,
                                         trust_remote_code=True)

llm = LLM(
    model=model_name,
    max_model_len=4096,
    max_num_seqs=2,
    trust_remote_code=True,
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)
# NOTE The stop_token_ids are different for various versions of MiniCPM-V
# 2.0
# stop_token_ids = [tokenizer.eos_id]

# 2.5
# stop_token_ids = [tokenizer.eos_id, tokenizer.eot_id]

# 2.6 / o2.6
stop_tokens = ['<|im_end|>', '<|endoftext|>']
stop_token_ids = [tokenizer.convert_tokens_to_ids(i) for i in stop_tokens]

modality_placeholder = {

    "image": "<image>./</image>",
    "video": "<video>./</video>",
}

prompts = [
    tokenizer.apply_chat_template(
        [{

            'role': 'user',
            'content': f"{
                modality_placeholder[modality]}\n{
                question}"
        }],
        tokenize=False,
        add_generation_prompt=True) for question in questions
    ]
return llm, prompts, stop_token_ids

```

```

def run_minicpmo(questions: list[str], modality: str):
    return run_minicpmv_base(questions, modality, "openbmb/MiniCPM-o-2_6")

def run_minicpmv(questions: list[str], modality: str):
    return run_minicpmv_base(questions, modality, "openbmb/MiniCPM-V-2_6")

# Llama 3.2
def run_llama(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "meta-llama/Llama-3.2-11B-Vision-Instruct"

    # Note: The default setting of max_num_seqs (256) and
    # max_model_len (131072) for this model may cause OOM.
    # You may lower either to run this example on lower-end GPUs.

    # The configuration below has been confirmed to launch on a single L40 GPU.
    llm = LLM(
        model=model_name,
        max_model_len=4096,
        max_num_seqs=16,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    messages = [{

        "role":
        "user",
        "content": [{

            "type": "image"

        }, {

            "type": "text",
            "text": question

        }]
    } for question in questions]
    prompts = tokenizer.apply_chat_template(messages,
                                           add_generation_prompt=True,
                                           tokenize=False)

    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

[illegible]

```

        stop_token_ids = None
    return llm, prompts, stop_token_ids

# PaliGemma
def run_paligemma(question: str, modality: str):
    assert modality == "image"

    # PaliGemma has special prompt format for VQA
    prompt = ["caption en"]
    llm = LLM(model="google/paligemma-3b-mix-224",
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompt, stop_token_ids

# PaliGemma 2
def run_paligemma2(question: str, modality: str):
    assert modality == "image"

    # PaliGemma 2 has special prompt format for VQA
    prompt = ["caption en"]
    llm = LLM(model="google/paligemma2-3b-ft-docci-448",
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompt, stop_token_ids

# Phi-3-Vision
def run_phi3v(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [
        f"<|user|>\n<|image_1|>\n{
            question}<|end|>\n<|assistant|>\n"
        for question in questions
    ]

    # num_crops is an override kwarg to the multimodal image processor;
    # For some models, e.g., Phi-3.5-vision-instruct, it is recommended
    # to use 16 for single frame scenarios, and 4 for multi-frame.
    #
    # Generally speaking, a larger value for num_crops results in more
    # tokens per image instance, because it may scale the image more in
    # the image preprocessing. Some references in the model docs and the
    # formula for image tokens after the preprocessing
    # transform can be found below.

```

```

# https://huggingface.co/microsoft/Phi-3.5-vision-instruct#Loading-the-model-locally
# https://huggingface.co/microsoft/Phi-3.5-vision-instruct/blob/main/processing_phi3_
v.py#L194
llm = LLM(
    model="microsoft/Phi-3.5-vision-instruct",
    trust_remote_code=True,
    max_model_len=4096,
    max_num_seqs=2,
    # Note - mm_processor_kwargs can also be passed to generate/chat calls
    mm_processor_kwargs={
        "num_crops": 16},
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)
stop_token_ids = None
return llm, prompts, stop_token_ids

# Phi-4-multimodal-instruct
def run_phi4mm(questions: list[str], modality: str):
    """
    Phi-4-multimodal-instruct supports both image and audio inputs. Here, we
    show how to process image inputs.
    """
    assert modality == "image"
    model_path = snapshot_download("microsoft/Phi-4-multimodal-instruct")
    # Since the vision-lora and speech-lora co-exist with the base model,
    # we have to manually specify the path of the lora weights.
    vision_lora_path = os.path.join(model_path, "vision-lora")
    prompts = [
        f"<|user|><|image_1|>{
            question}<|end|><|assistant|>"
        for question in questions
    ]
    llm = LLM(
        model=model_path,
        trust_remote_code=True,
        max_model_len=4096,
        max_num_seqs=2,
        enable_lora=True,
        max_lora_rank=320,
        lora_extra_vocab_size=0,
    )
    lora_request = LoRARequest("vision", 1, vision_lora_path)
    # To maintain code compatibility in this script, we add LoRA here.
    llm.llm_engine.add_lora(lora_request=lora_request)
    # You can also add LoRA using:
    # llm.generate(prompts, lora_request=lora_request, ...)

```

```

        stop_token_ids = None
        return llm, prompts, stop_token_ids

# Pixtral HF-format
def run_pixtral_hf(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "mistral-community/pixtral-12b"

    # NOTE: Need L40 (or equivalent) to avoid OOM
    llm = LLM(
        model=model_name,
        max_model_len=8192,
        max_num_seqs=2,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    prompts = [f"<s>[INST]{
        question}\n[IMG][/INST]" for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Qwen
def run_qwen_vl(questions: list[str], modality: str):
    assert modality == "image"

    llm = LLM(
        model="Qwen/Qwen-VL",
        trust_remote_code=True,
        max_model_len=1024,
        max_num_seqs=2,
        hf_overrides={
            "architectures": ["QwenVLForConditionalGeneration"]},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    prompts = [f"{
        question}Picture 1: <img></img>\n" for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Qwen2-VL
def run_qwen2_vl(questions: list[str], modality: str):

```



```

model_name = "Qwen/Qwen2-VL-7B-Instruct"

llm = LLM(
    model=model_name,
    max_model_len=4096,
    max_num_seqs=5,
    # Note - mm_processor_kwargs can also be passed to generate/chat calls
    mm_processor_kwargs={
        "min_pixels": 28 * 28,
        "max_pixels": 1280 * 28 * 28,
    },
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)

if modality == "image":
    placeholder = "<|image_pad|>"
elif modality == "video":
    placeholder = "<|video_pad|>"

prompts = [
    ("<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
     f"<|im_start|>user\n<|vision_start|>{
         placeholder}<|vision_end|>"
     f"{
         question}<|im_end|>\n"
     "<|im_start|>assistant\n") for question in questions
]
stop_token_ids = None
return llm, prompts, stop_token_ids

# Qwen2.5-VL
def run_qwen2_5_vl(questions: list[str], modality: str):

    model_name = "Qwen/Qwen2.5-VL-3B-Instruct"

    llm = LLM(
        model=model_name,
        max_model_len=4096,
        max_num_seqs=5,
        mm_processor_kwargs={
            "min_pixels": 28 * 28,
            "max_pixels": 1280 * 28 * 28,
            "fps": 1,

```

```

        },
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    if modality == "image":
        placeholder = "<|image_pad|>"
    elif modality == "video":
        placeholder = "<|video_pad|>"

    prompts = [
        ("<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
         f"<|im_start|>user\n<|vision_start|>{
             placeholder}<|vision_end|>"
         f"{
             question}<|im_end|>\n"
         "<|im_start|>assistant\n") for question in questions
    ]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

model_example_map = {

    "aria": run_aria,
    "blip-2": run_blip2,
    "chameleon": run_chameleon,
    "deepseek_v1_v2": run_deepseek_v12,
    "florence2": run_florence2,
    "fuyu": run_fuyu,
    "gemma3": run_gemma3,
    "glm4v": run_glm4v,
    "h2ovl_chat": run_h2ovl,
    "idefics3": run_idefics3,
    "internvl_chat": run_internvl,
    "llava": run_llava,
    "llava-next": run_llava_next,
    "llava-next-video": run_llava_next_video,
    "llava-onevision": run_llava_onevision,
    "mantis": run_mantis,
    "minicpmo": run_minicpmo,
    "minicpmv": run_minicpmv,
    "mllama": run_mllama,
    "molmo": run_molmo,
    "NVLM_D": run_nvlm_d,
    "paligemma": run_paligemma,
    "paligemma2": run_paligemma2,
    "phi3_v": run_phi3v,
    "phi4 mm": run_phi4mm,

```

```

        "pixtral_hf": run_pixtral_hf,
        "qwen_v1": run_qwen_v1,
        "qwen2_v1": run_qwen2_v1,
        "qwen2_5_v1": run_qwen2_5_v1,
    }

def get_multi_modal_input(args):
    """
    return {
        "data": image or video,
        "question": question,
    }
    """
    if args.modality == "image":
        # Input image and question
        image = ImageAsset("cherry_blossom") \
            .pil_image.convert("RGB")
        img_questions = [
            "What is the content of this image?",
            "Describe the content of this image in detail.",
            "What's in the image?",
            "Where is this image taken?",
        ]

        return {

            "data": image,
            "questions": img_questions,
        }

    if args.modality == "video":
        # Input video and question
        video = VideoAsset(name="sample_demo_1.mp4",
                           num_frames=args.num_frames).np_ndarrays
        vid_questions = ["Why is this video funny?"]

        return {

            "data": video,
            "questions": vid_questions,
        }

    msg = f"Modality {
        args.modality} is not supported."
    raise ValueError(msg)

```

```

def apply_image_repeat(image_repeat_prob, num_prompts, data,
                        prompts: list[str], modality):
    """Repeats images with provided probability of "image_repeat_prob".
    Used to simulate hit/miss for the MM preprocessor cache.
    """
    assert (image_repeat_prob <= 1.0 and image_repeat_prob >= 0)
    no_yes = [0, 1]
    probs = [1.0 - image_repeat_prob, image_repeat_prob]

    inputs = []
    cur_image = data
    for i in range(num_prompts):
        if image_repeat_prob is not None:
            res = random.choices(no_yes, probs)[0]
            if res == 0:
                # No repeat => Modify one pixel
                cur_image = cur_image.copy()
                new_val = (i // 256 // 256, i // 256, i % 256)
                cur_image.putpixel((0, 0), new_val)

        inputs.append({
            "prompt": prompts[i % len(prompts)],
            "multi_modal_data": {
                "image": cur_image
            }
        })

    return inputs

def main(args):
    model = args.model_type
    if model not in model_example_map:
        raise ValueError(f"Model type {
            model} is not supported.")

    modality = args.modality
    mm_input = get_multi_modal_input(args)
    data = mm_input["data"]
    questions = mm_input["questions"]

    llm, prompts, stop_token_ids = model_example_map[model](questions,
                                                                modality)

    # Don't want to check the flag multiple times, so just hijack `prompts`.
    prompts = prompts if args.use_different_prompt_per_request else [

```

```

    prompts = prompts if args.use_different_prompt_per_request else [
        prompts[0]
    ]

    # We set temperature to 0.2 so that outputs can be different
    # even when all prompts are identical when running batch inference.
    sampling_params = SamplingParams(temperature=0.2,
                                     max_tokens=64,
                                     stop_token_ids=stop_token_ids)

    assert args.num_prompts > 0
    if args.num_prompts == 1:
        # Single inference
        inputs = {
            "prompt": prompts[0],
            "multi_modal_data": {
                "image": data
            },
        }
    else:
        # Batch inference
        if args.image_repeat_prob is not None:
            # Repeat images with specified probability of "image_repeat_prob"
            inputs = apply_image_repeat(args.image_repeat_prob,
                                       args.num_prompts, data, prompts,
                                       modality)
        else:
            # Use the same image for all prompts
            inputs = [{
                "prompt": prompts[i % len(prompts)],
                "multi_modal_data": {
                    "image": data
                },
            } for i in range(args.num_prompts)]

    if args.time_generate:
        import time
        start_time = time.time()
        outputs = llm.generate(inputs, sampling_params=sampling_params)
        elapsed_time = time.time() - start_time
        print("-- generate time = {}".format(elapsed_time))

    else:
        outputs = llm.generate(inputs, sampling_params=sampling_params)

```

```

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

if __name__ == "__main__":
    parser = FlexibleArgumentParser(
        description='Demo on using vLLM for offline inference with '
        'vision language models for text generation')
    parser.add_argument('--model-type',
                        '-m',
                        type=str,
                        default="llava",
                        choices=model_example_map.keys(),
                        help='Huggingface "model_type".')
    parser.add_argument('--num-prompts',
                        type=int,
                        default=4,
                        help='Number of prompts to run.')
    parser.add_argument('--modality',
                        type=str,
                        default="image",
                        choices=['image', 'video'],
                        help='Modality of the input.')
    parser.add_argument('--num-frames',
                        type=int,
                        default=16,
                        help='Number of frames to extract from the video.')

    parser.add_argument(
        '--image-repeat-prob',
        type=float,
        default=None,
        help='Simulates the hit-ratio for multi-modal preprocessor cache'
        ' (if enabled)')

    parser.add_argument(
        '--disable-mm-preprocessor-cache',
        action='store_true',
        help='If True, disables caching of multi-modal preprocessor/mapper.')

    parser.add_argument(
        '--time-generate',
        action='store_true',
        help='If True, then print the total generate() call time')

```

```

parser.add_argument(
    '--use-different-prompt-per-request',
    action='store_true',
    help='If True, then use different prompt (with the same multi-modal '
    'data) for each request.')

args = parser.parse_args()
main(args)

```

如果需要在同一个文本提示中替换多张图像，可以传入一个图像列表，如下所示：

```

# Specify the maximum number of frames per video to be 4. This can be changed.
llm = LLM("Qwen/Qwen2-VL-2B-Instruct", limit_mm_per_prompt={
    "image": 4})

# Create the request payload.
video_frames = ... # Load your video making sure it only has the number of frames specified earlier.
message = {

    "role": "user",
    "content": [
        {
            "type": "text", "text": "Describe this set of frames. Consider the frames to be a part of the same video."},
    ],
}

for i in range(len(video_frames)):
    base64_image = encode_image(video_frames[i]) # base64 encoding.
    new_image = {
        "type": "image_url", "image_url": {
            "url": f"data:image/jpeg;base64,{
                base64_image}"}}
    message["content"].append(new_image)

# Perform inference and Log output.
outputs = llm.chat([message])

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

```

1.2 视频输入

你可以直接将 NumPy 数组列表传递给多模态字典中的 `'video'` 字段，而无需使用多图像输入。

完整示例如下： `vision_language.py`

"""

This example shows how to use vLLM for running offline inference with the correct prompt format on vision language models for text generation.

For most models, the prompt format should follow corresponding examples on HuggingFace model repository.

"""

import os

import random

from huggingface_hub import snapshot_download

from transformers import AutoTokenizer

from vllm import LLM, SamplingParams

from vllm.assets.image import ImageAsset

from vllm.assets.video import VideoAsset

from vllm.lora.request import LoRARequest

from vllm.utils import FlexibleArgumentParser

NOTE: The default `max\_num\_seqs` and `max\_model\_len` may result in OOM on

Lower-end GPUs.

Unless specified, these settings have been tested to work on a single L4.

Aria

def run_aria(questions: list[str], modality: str):

assert modality == "image"

model_name = "rhymes-ai/Aria"

NOTE: Need L40 (or equivalent) to avoid OOM

llm = LLM(model=model_name,

max_model_len=4096,

max_num_seqs=2,

dtype="bfloat16",

disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)

prompts = [(f"<|im_start|>user\n<fim_prefix><|img|><fim_suffix>{
question}")

"<|im_end|>\n<|im_start|>assistant\n")

for question in questions]

stop_token_ids = [93532, 93653, 944, 93421, 1019, 93653, 93519]

return llm, prompts, stop_token_ids

BLIP-2

def run_blip2(questions: list[str], modality: str):


```

def run_blip2(questions: list[str], modality: str):
    assert modality == "image"

    # BLIP-2 prompt format is inaccurate on HuggingFace model repository.
    # See https://huggingface.co/Salesforce/blip2-opt-2.7b/discussions/15#64ff02f3f8cf9e4
    # f5b038262 #noqa
    prompts = [f"Question: {
                question} Answer:" for question in questions]
    llm = LLM(model="Salesforce/blip2-opt-2.7b",
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Chameleon
def run_chameleon(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"{
                question}<image>" for question in questions]
    llm = LLM(model="facebook/chameleon-7b",
              max_model_len=4096,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Deepseek-VL2
def run_deepseek_vl2(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "deepseek-ai/deepseek-vl2-tiny"

    llm = LLM(model=model_name,
              max_model_len=4096,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
              hf_overrides={
                  "architectures": ["DeepseekVLV2ForCausalLM"]})

    prompts = [
        f"<|User|>: <image>\n{
            question}\n\n<|Assistant|>:"
        for question in questions
    ]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

```

# Florence2
def run_florence2(question: str, modality: str):
    assert modality == "image"

    llm = LLM(model="microsoft/Florence-2-large",
              tokenizer="facebook/bart-large",
              max_num_seqs=8,
              trust_remote_code=True,
              dtype="bfloat16",
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)

    prompt = "<MORE_DETAILED_CAPTION>"
    stop_token_ids = None
    return llm, prompt, stop_token_ids

```

```

# Fuyu
def run_fuyu(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"{
        question}\n" for question in questions]
    llm = LLM(model="adept/fuyu-8b",
              max_model_len=2048,
              max_num_seqs=2,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

```

# Gemma 3
def run_gemma3(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "google/gemma-3-4b-it"

    llm = LLM(
        model=model_name,
        max_model_len=2048,
        max_num_seqs=2,
        # Default is False; setting it to True is not supported in V1 yet
        mm_processor_kwargs={
            "do_pan_and_scan": True},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

```

```

prompts = [f"<bos><start of turn>user\n"

```

```

        f"<start_of_image>{
        question}<end_of_turn>\n"
        "<start_of_turn>model\n") for question in questions]
stop_token_ids = None
return llm, prompts, stop_token_ids

# GLM-4v
def run_glm4v(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "THUDM/glm-4v-9b"

    llm = LLM(model=model_name,
               max_model_len=2048,
               max_num_seqs=2,
               trust_remote_code=True,
               enforce_eager=True,
               hf_overrides={
                   "architectures": ["GLM4VForCausalLM"]},
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)

    prompts = [
        f"<|user|>\n<|begin_of_image|><|endoftext|><|end_of_image|>\n"
        {
            question}<|assistant|>" for question in questions
    ]

    stop_token_ids = [151329, 151336, 151338]
    return llm, prompts, stop_token_ids

# H2OVL-Mississippi
def run_h2ovl(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "h2oai/h2ovl-mississippi-800m"

    llm = LLM(
        model=model_name,
        trust_remote_code=True,
        max_model_len=8192,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    tokenizer = AutoTokenizer.from_pretrained(model_name,
                                              trust_remote_code=True)

    messages = [[{

```

```

        'role': 'user',
        'content': f"<image>\n{
            question}"
    }] for question in questions]
    prompts = tokenizer.apply_chat_template(messages,
                                           tokenize=False,
                                           add_generation_prompt=True)

    # Stop tokens for H2OVL-Mississippi
    # https://huggingface.co/h2oai/h2ovl-mississippi-800m
    stop_token_ids = [tokenizer.eos_token_id]
    return llm, prompts, stop_token_ids

# Idefics3-8B-LLama3
def run_idefics3(questions: list[str], modality: str):
    assert modality == "image"
    model_name = "HuggingFaceM4/Idfics3-8B-LLama3"

    llm = LLM(
        model=model_name,
        max_model_len=8192,
        max_num_seqs=2,
        enforce_eager=True,
        # if you are running out of memory, you can reduce the "longest_edge".
        # see: https://huggingface.co/HuggingFaceM4/Idfics3-8B-LLama3#model-optimization

        mm_processor_kwargs={

            "size": {

                "longest_edge": 3 * 364

            },

        },
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )
    prompts = [
        f"<|begin_of_text|>User:<image>{
            question}<end_of_utterance>\nAssistant:"
    ] for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# InternVL
def run_internvl(questions: list[str], modality: str):
    # TODO: Implement InternVL logic

```

```

assert modality == "image"

model_name = "OpenGVLab/InternVL2-2B"

llm = LLM(
    model=model_name,
    trust_remote_code=True,
    max_model_len=4096,
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)

tokenizer = AutoTokenizer.from_pretrained(model_name,
                                          trust_remote_code=True)

messages = [[{
    'role': 'user',
    'content': f"<image>\n{
        question}"
}] for question in questions]
prompts = tokenizer.apply_chat_template(messages,
                                       tokenize=False,
                                       add_generation_prompt=True)

# Stop tokens for InternVL
# models variants may have different stop tokens
# please refer to the model card for the correct "stop words":
# https://huggingface.co/OpenGVLab/InternVL2-2B/blob/main/conversation.py
stop_tokens = ["<|endoftext|>", "<|im_start|>", "<|im_end|>", "<|end|>"]
stop_token_ids = [tokenizer.convert_tokens_to_ids(i) for i in stop_tokens]
return llm, prompts, stop_token_ids

# LLaVA-1.5
def run_llava(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [
        f"USER: <image>\n{
            question}\nASSISTANT:" for question in questions
    ]

    llm = LLM(model="llava-hf/llava-1.5-7b-hf",
              max_model_len=4096,
              disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

```

# LLaVA-1.6/LLaVA-NeXT
def run_llava_next(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [f"[INST] <image>\n{
                question} [/INST]" for question in questions]
    llm = LLM(model="llava-hf/llava-v1.6-mistral-7b-hf",
               max_model_len=8192,
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# LLaVA-NeXT-Video
# Currently only support for video input
def run_llava_next_video(questions: list[str], modality: str):
    assert modality == "video"

    prompts = [
        f"USER: <video>\n{
            question} ASSISTANT:" for question in questions
    ]
    llm = LLM(model="llava-hf/LLaVA-NeXT-Video-7B-hf",
               max_model_len=8192,
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# LLaVA-OneVision
def run_llava_onevision(questions: list[str], modality: str):

    if modality == "video":
        prompts = [
            f"<|im_start|>user <video>\n{
                question}<|im_end|> \
            <|im_start|>assistant\n" for question in questions
        ]

    elif modality == "image":
        prompts = [
            f"<|im_start|>user <image>\n{
                question}<|im_end|> \
            <|im_start|>assistant\n" for question in questions
        ]

    llm = LLM(model="llava-hf/llava-onevision-qwen2-7b-ov-hf",

```

```

        max_model_len=16384,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Mantis
def run_mantis(questions: list[str], modality: str):
    assert modality == "image"

    llama3_template = '<|start_header_id|>user<|end_header_id|>\n\n{<|eot_id|><|start_header_id|>assistant<|end_header_id|>\n\n' # noqa: E501
    prompts = [
        llama3_template.format(f"{
            question}\n\n<image>")
        for question in questions
    ]

    llm = LLM(
        model="TIGER-Lab/Mantis-8B-siglip-llama3",
        max_model_len=4096,
        hf_overrides={
            "architectures": ["MantisForConditionalGeneration"]},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )
    stop_token_ids = [128009]
    return llm, prompts, stop_token_ids

# MiniCPM-V
def run_minicpmv_base(questions: list[str], modality: str, model_name):
    assert modality in ["image", "video"]
    # If you want to use `MiniCPM-o-2_6` with audio inputs, check `audio_language.py` # noqa

    # 2.0
    # The official repo doesn't work yet, so we need to use a fork for now
    # For more details, please see: See: https://github.com/vllm-project/vllm/pull/4087#issuecomment-2250397630 # noqa
    # model_name = "HwvH/MiniCPM-V-2"

    # 2.5
    # model_name = "openbmb/MiniCPM-LLama3-V-2_5"

    # 2.6
    # model_name = "openbmb/MiniCPM-V-2_6"
    # o2.6

```

```

# modality supports
# 2.0: image
# 2.5: image
# 2.6: image, video
# o2.6: image, video, audio
# model_name = "openbmb/MiniCPM-o-2_6"
tokenizer = AutoTokenizer.from_pretrained(model_name,
                                         trust_remote_code=True)

llm = LLM(
    model=model_name,
    max_model_len=4096,
    max_num_seqs=2,
    trust_remote_code=True,
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)
# NOTE The stop_token_ids are different for various versions of MiniCPM-V
# 2.0
# stop_token_ids = [tokenizer.eos_id]

# 2.5
# stop_token_ids = [tokenizer.eos_id, tokenizer.eot_id]

# 2.6 / o2.6
stop_tokens = ['<|im_end|>', '<|endoftext|>']
stop_token_ids = [tokenizer.convert_tokens_to_ids(i) for i in stop_tokens]

modality_placeholder = {

    "image": "<image>./</image>",
    "video": "<video>./</video>",
}

prompts = [
    tokenizer.apply_chat_template(
        [{

            'role': 'user',
            'content': f"{
                modality_placeholder[modality]}\n{
                question}"
        }],
        tokenize=False,
        add_generation_prompt=True) for question in questions
    ]
return llm, prompts, stop_token_ids

```



```

def run_minicpmo(questions: list[str], modality: str):
    return run_minicpmv_base(questions, modality, "openbmb/MiniCPM-o-2_6")

def run_minicpmv(questions: list[str], modality: str):
    return run_minicpmv_base(questions, modality, "openbmb/MiniCPM-V-2_6")

# Llama 3.2
def run_llama(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "meta-llama/Llama-3.2-11B-Vision-Instruct"

    # Note: The default setting of max_num_seqs (256) and
    # max_model_len (131072) for this model may cause OOM.
    # You may lower either to run this example on lower-end GPUs.

    # The configuration below has been confirmed to launch on a single L40 GPU.
    llm = LLM(
        model=model_name,
        max_model_len=4096,
        max_num_seqs=16,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    messages = [{

        "role":
        "user",
        "content": [{

            "type": "image"

        }, {

            "type": "text",
            "text": question

        }]
    } for question in questions]
    prompts = tokenizer.apply_chat_template(messages,
                                           add_generation_prompt=True,
                                           tokenize=False)

    stop_token_ids = None
    return llm, prompts, stop_token_ids

```

[illegible]

```

stop_token_ids = None
return llm, prompts, stop_token_ids

# PaliGemma
def run_paligemma(question: str, modality: str):
    assert modality == "image"

    # PaliGemma has special prompt format for VQA
    prompt = ["caption en"]
    llm = LLM(model="google/paligemma-3b-mix-224",
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompt, stop_token_ids

# PaliGemma 2
def run_paligemma2(question: str, modality: str):
    assert modality == "image"

    # PaliGemma 2 has special prompt format for VQA
    prompt = ["caption en"]
    llm = LLM(model="google/paligemma2-3b-ft-docci-448",
               disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache)
    stop_token_ids = None
    return llm, prompt, stop_token_ids

# Phi-3-Vision
def run_phi3v(questions: list[str], modality: str):
    assert modality == "image"

    prompts = [
        f"<|user|>\n<|image_1|>\n{
            question}<|end|>\n<|assistant|>\n"
        for question in questions
    ]

    # num_crops is an override kwarg to the multimodal image processor;
    # For some models, e.g., Phi-3.5-vision-instruct, it is recommended
    # to use 16 for single frame scenarios, and 4 for multi-frame.
    #
    # Generally speaking, a larger value for num_crops results in more
    # tokens per image instance, because it may scale the image more in
    # the image preprocessing. Some references in the model docs and the
    # formula for image tokens after the preprocessing
    # transform can be found below.

```

```

#
# https://huggingface.co/microsoft/Phi-3.5-vision-instruct#loading-the-model-locally
# https://huggingface.co/microsoft/Phi-3.5-vision-instruct/blob/main/processing_phi3_
v.py#L194
llm = LLM(
    model="microsoft/Phi-3.5-vision-instruct",
    trust_remote_code=True,
    max_model_len=4096,
    max_num_seqs=2,
    # Note - mm_processor_kwargs can also be passed to generate/chat calls
    mm_processor_kwargs={
        "num_crops": 16},
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)
stop_token_ids = None
return llm, prompts, stop_token_ids

# Phi-4-multimodal-instruct
def run_phi4mm(questions: list[str], modality: str):
    """
    Phi-4-multimodal-instruct supports both image and audio inputs. Here, we
    show how to process image inputs.
    """
    assert modality == "image"
    model_path = snapshot_download("microsoft/Phi-4-multimodal-instruct")
    # Since the vision-lora and speech-lora co-exist with the base model,
    # we have to manually specify the path of the lora weights.
    vision_lora_path = os.path.join(model_path, "vision-lora")
    prompts = [
        f"<|user|><|image_1|>{
            question}<|end|><|assistant|>"
        for question in questions
    ]
    llm = LLM(
        model=model_path,
        trust_remote_code=True,
        max_model_len=4096,
        max_num_seqs=2,
        enable_lora=True,
        max_lora_rank=320,
        lora_extra_vocab_size=0,
    )
    lora_request = LoRARequest("vision", 1, vision_lora_path)
    # To maintain code compatibility in this script, we add LoRA here.
    llm.llm_engine.add_lora(lora_request=lora_request)
    # You can also add LoRA using:
    # llm.generate(prompts, lora_request=lora_request, ...)

```

```

        stop_token_ids = None
    return llm, prompts, stop_token_ids

# Pixtral HF-format
def run_pixtral_hf(questions: list[str], modality: str):
    assert modality == "image"

    model_name = "mistral-community/pixtral-12b"

    # NOTE: Need L40 (or equivalent) to avoid OOM
    llm = LLM(
        model=model_name,
        max_model_len=8192,
        max_num_seqs=2,
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    prompts = [f"<s>[INST]{
        question}\n[IMG]/[INST]" for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Qwen
def run_qwen_vl(questions: list[str], modality: str):
    assert modality == "image"

    llm = LLM(
        model="Qwen/Qwen-VL",
        trust_remote_code=True,
        max_model_len=1024,
        max_num_seqs=2,
        hf_overrides={
            "architectures": ["QwenVLForConditionalGeneration"]},
        disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
    )

    prompts = [f"{
        question}Picture 1: <img></img>\n" for question in questions]
    stop_token_ids = None
    return llm, prompts, stop_token_ids

# Qwen2-VL
def run_qwen2_vl(questions: list[str], modality: str):

```

```

model_name = "Qwen/Qwen2-VL-7B-Instruct"

llm = LLM(
    model=model_name,
    max_model_len=4096,
    max_num_seqs=5,
    # Note - mm_processor_kwargs can also be passed to generate/chat calls
    mm_processor_kwargs={

        "min_pixels": 28 * 28,
        "max_pixels": 1280 * 28 * 28,
    },
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)

if modality == "image":
    placeholder = "<|image_pad|>"
elif modality == "video":
    placeholder = "<|video_pad|>"

prompts = [
    ("<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
     f"<|im_start|>user\n<|vision_start|>{
         placeholder}<|vision_end|>"
     f"{
         question}<|im_end|>\n"
     "<|im_start|>assistant\n") for question in questions
]
stop_token_ids = None
return llm, prompts, stop_token_ids

# Qwen2.5-VL
def run_qwen2_5_vl(questions: list[str], modality: str):

    model_name = "Qwen/Qwen2.5-VL-3B-Instruct"

    llm = LLM(
        model=model_name,
        max_model_len=4096,
        max_num_seqs=5,
        mm_processor_kwargs={

            "min_pixels": 28 * 28,
            "max_pixels": 1280 * 28 * 28,
            "fps": 1,

```

```

    },
    disable_mm_preprocessor_cache=args.disable_mm_preprocessor_cache,
)

if modality == "image":
    placeholder = "<|image_pad|>"
elif modality == "video":
    placeholder = "<|video_pad|>"

prompts = [
    ("<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
     f"<|im_start|>user\n<|vision_start|>{
         placeholder}<|vision_end|>"
     f"{
         question}<|im_end|>\n"
     "<|im_start|>assistant\n") for question in questions
]
stop_token_ids = None
return llm, prompts, stop_token_ids

```

model_example_map = {

```

    "aria": run_aria,
    "blip-2": run_blip2,
    "chameleon": run_chameleon,
    "deepseek_v1_v2": run_deepseek_v12,
    "florence2": run_florence2,
    "fuyu": run_fuyu,
    "gemma3": run_gemma3,
    "glm4v": run_glm4v,
    "h2ovl_chat": run_h2ovl,
    "idefics3": run_idefics3,
    "internvl_chat": run_internvl,
    "llava": run_llava,
    "llava-next": run_llava_next,
    "llava-next-video": run_llava_next_video,
    "llava-onevision": run_llava_onevision,
    "mantis": run_mantis,
    "minicpmo": run_minicpmo,
    "minicpmv": run_minicpmv,
    "mllama": run_mllama,
    "mo1mo": run_mo1mo,
    "NVLM_D": run_nv1m_d,
    "paligemma": run_paligemma,
    "paligemma2": run_paligemma2,
    "phi3_v": run_phi3v,
    "phi4 mm": run_phi4mm,

```

```

        "pixtral_hf": run_pixtral_hf,
        "qwen_v1": run_qwen_v1,
        "qwen2_v1": run_qwen2_v1,
        "qwen2_5_v1": run_qwen2_5_v1,
    }

def get_multi_modal_input(args):
    """
    return {
        "data": image or video,
        "question": question,
    }
    """
    if args.modality == "image":
        # Input image and question
        image = ImageAsset("cherry_blossom") \
            .pil_image.convert("RGB")
        img_questions = [
            "What is the content of this image?",
            "Describe the content of this image in detail.",
            "What's in the image?",
            "Where is this image taken?",
        ]

        return {

            "data": image,
            "questions": img_questions,
        }

    if args.modality == "video":
        # Input video and question
        video = VideoAsset(name="sample_demo_1.mp4",
                           num_frames=args.num_frames).np_ndarrays
        vid_questions = ["Why is this video funny?"]

        return {

            "data": video,
            "questions": vid_questions,
        }

    msg = f"Modality {
        args.modality} is not supported."
    raise ValueError(msg)

```



```

def apply_image_repeat(image_repeat_prob, num_prompts, data,
                        prompts: list[str], modality):
    """Repeats images with provided probability of "image_repeat_prob".
    Used to simulate hit/miss for the MM preprocessor cache.
    """
    assert (image_repeat_prob <= 1.0 and image_repeat_prob >= 0)
    no_yes = [0, 1]
    probs = [1.0 - image_repeat_prob, image_repeat_prob]

    inputs = []
    cur_image = data
    for i in range(num_prompts):
        if image_repeat_prob is not None:
            res = random.choices(no_yes, probs)[0]
            if res == 0:
                # No repeat => Modify one pixel
                cur_image = cur_image.copy()
                new_val = (i // 256 // 256, i // 256, i % 256)
                cur_image.putpixel((0, 0), new_val)

        inputs.append({
            "prompt": prompts[i % len(prompts)],
            "multi_modal_data": {
                "image": cur_image
            }
        })

    return inputs

def main(args):
    model = args.model_type
    if model not in model_example_map:
        raise ValueError(f"Model type {
            model} is not supported.")

    modality = args.modality
    mm_input = get_multi_modal_input(args)
    data = mm_input["data"]
    questions = mm_input["questions"]

    llm, prompts, stop_token_ids = model_example_map[model](questions,
                                                                modality)

    # Don't want to check the flag multiple times, so just hijack `prompts`.
    prompts = prompts if args.use_different_prompt_per_request else [

```

```

prompts = prompts if args.use_different_prompt_per_request else [
    prompts[0]
]

# We set temperature to 0.2 so that outputs can be different
# even when all prompts are identical when running batch inference.
sampling_params = SamplingParams(temperature=0.2,
                                max_tokens=64,
                                stop_token_ids=stop_token_ids)

assert args.num_prompts > 0
if args.num_prompts == 1:
    # Single inference
    inputs = {
        "prompt": prompts[0],
        "multi_modal_data": {
            modality: data
        },
    }
else:
    # Batch inference
    if args.image_repeat_prob is not None:
        # Repeat images with specified probability of "image_repeat_prob"
        inputs = apply_image_repeat(args.image_repeat_prob,
                                    args.num_prompts, data, prompts,
                                    modality)
    else:
        # Use the same image for all prompts
        inputs = [{
            "prompt": prompts[i % len(prompts)],
            "multi_modal_data": {
                modality: data
            },
        } for i in range(args.num_prompts)]

if args.time_generate:
    import time
    start_time = time.time()
    outputs = llm.generate(inputs, sampling_params=sampling_params)
    elapsed_time = time.time() - start_time
    print("-- generate time = {}".format(elapsed_time))

else:
    outputs = llm.generate(inputs, sampling_params=sampling_params)

```

```

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

if __name__ == "__main__":
    parser = FlexibleArgumentParser(
        description='Demo on using vLLM for offline inference with '
        'vision language models for text generation')
    parser.add_argument('--model-type',
                        '-m',
                        type=str,
                        default="llava",
                        choices=model_example_map.keys(),
                        help='Huggingface "model_type".')
    parser.add_argument('--num-prompts',
                        type=int,
                        default=4,
                        help='Number of prompts to run.')
    parser.add_argument('--modality',
                        type=str,
                        default="image",
                        choices=['image', 'video'],
                        help='Modality of the input.')
    parser.add_argument('--num-frames',
                        type=int,
                        default=16,
                        help='Number of frames to extract from the video.')

    parser.add_argument(
        '--image-repeat-prob',
        type=float,
        default=None,
        help='Simulates the hit-ratio for multi-modal preprocessor cache'
        ' (if enabled)')

    parser.add_argument(
        '--disable-mm-preprocessor-cache',
        action='store_true',
        help='If True, disables caching of multi-modal preprocessor/mapper.')

    parser.add_argument(
        '--time-generate',
        action='store_true',
        help='If True, then print the total generate() call time')

    parser.add_argument(

```

```

parser.add_argument(
    '--use-different-prompt-per-request',
    action='store_true',
    help='If True, then use different prompt (with the same multi-modal '
    'data) for each request.')

args = parser.parse_args()
main(args)

```

1.3 音频输入

你可以将一个元组 `(array, sampling_rate)` 传递给多模态字典中的 `'audio'` 字段。

完整示例如下: `audio_language.py`

```

# SPDX-License-Identifier: Apache-2.0
"""
This example shows how to use vLLM for running offline inference
with the correct prompt format on audio language models.

For most models, the prompt format should follow corresponding examples
on HuggingFace model repository.
"""
import os

from huggingface_hub import snapshot_download
from transformers import AutoTokenizer

from vllm import LLM, SamplingParams
from vllm.assets.audio import AudioAsset
from vllm.lora.request import LoRARequest
from vllm.utils import FlexibleArgumentParser

audio_assets = [AudioAsset("mary_had_lamb"), AudioAsset("winning_call")]
question_per_audio_count = {

    0: "What is 1+1?",
    1: "What is recited in the audio?",
    2: "What sport and what nursery rhyme are referenced?"
}

# NOTE: The default `max_num_seqs` and `max_model_len` may result in OOM on
# Lower-end GPUs.
# Unless specified, these settings have been tested to work on a single L4.

# MiniCPM-0
def run_minicpmo(question: str, audio_count: int):
    """
    """

```

```

model_name = "openbmb/MiniCPM-o-2_6"
tokenizer = AutoTokenizer.from_pretrained(model_name,
                                         trust_remote_code=True)

llm = LLM(model=model_name,
          trust_remote_code=True,
          max_model_len=4096,
          max_num_seqs=5,
          limit_mm_per_prompt={
              "audio": audio_count})

stop_tokens = ['<|im_end|>', '<|endoftext|>']
stop_token_ids = [tokenizer.convert_tokens_to_ids(i) for i in stop_tokens]

audio_placeholder = "<audio>.</audio>" * audio_count
audio_chat_template = "{% for message in messages %}{
    '<|im_start|>' + message['role'] + '\n' + message['content'] + '<|im_end|>'
+ '\n'}}{% endfor %}{% if add_generation_prompt %}{
    { '<|im_start|>assistant\n<|spk_bos|><|spk|><|spk_eos|><|tts_bos|>' }}{% endi
f %}" # noqa: E501
messages = [{

    'role': 'user',
    'content': f'{
        audio_placeholder}\n{
        question}'

}]
prompt = tokenizer.apply_chat_template(messages,
                                     tokenize=False,
                                     add_generation_prompt=True,
                                     chat_template=audio_chat_template)

return llm, prompt, stop_token_ids

# Phi-4-multimodal-instruct
def run_phi4mm(questions: str, audio_count: int):
    """
    Phi-4-multimodal-instruct supports both image and audio inputs. Here, we
    show how to process audio inputs.
    """
    model_path = snapshot_download("microsoft/Phi-4-multimodal-instruct")
    # Since the vision-lora and speech-lora co-exist with the base model,
    # we have to manually specify the path of the lora weights.
    speech_lora_path = os.path.join(model_path, "speech-lora")
    placeholders = "".join([f"<|audio_{
        i+1}|>" for i in range(audio_count)])

    prompts = f"<|user|>{
        placeholders}{

```

```

        questions}<|end|><|assistant|>"

llm = LLM(
    model=model_path,
    trust_remote_code=True,
    max_model_len=4096,
    max_num_seqs=2,
    enable_lora=True,
    max_lora_rank=320,
    lora_extra_vocab_size=0,
    limit_mm_per_prompt={
        "audio": audio_count},
)

lora_request = LoRARequest("speech", 1, speech_lora_path)
# To maintain code compatibility in this script, we add LoRA here.
llm.llm_engine.add_lora(lora_request=lora_request)
# You can also add LoRA using:
# llm.generate(prompts, lora_request=lora_request,...)

stop_token_ids = None
return llm, prompts, stop_token_ids

# Qwen2-Audio
def run_qwen2_audio(question: str, audio_count: int):
    model_name = "Qwen/Qwen2-Audio-7B-Instruct"

    llm = LLM(model=model_name,
               max_model_len=4096,
               max_num_seqs=5,
               limit_mm_per_prompt={
                   "audio": audio_count})

    audio_in_prompt = "".join([
        f"Audio {
            idx+1}: "
        f"<|audio_bos|><|AUDIO|><|audio_eos|>\n" for idx in range(audio_count)
    ])

    prompt = ("<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
              "<|im_start|>user\n"
              f"{
                  audio_in_prompt}{
                  question}<|im_end|>\n"
              "<|im_start|>assistant\n")

    stop_token_ids = None
    return llm, prompt, stop_token_ids

```

```

# Ultravox 0.5-1B
def run_ultravox(question: str, audio_count: int):
    model_name = "fixie-ai/ultravox-v0_5-llama-3_2-1b"

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    messages = [{
        'role': 'user',
        'content': "<|audio|>\n" * audio_count + question
    }]
    prompt = tokenizer.apply_chat_template(messages,
                                           tokenize=False,
                                           add_generation_prompt=True)

    llm = LLM(model=model_name,
               max_model_len=4096,
               max_num_seqs=5,
               trust_remote_code=True,
               limit_mm_per_prompt={
                   "audio": audio_count})
    stop_token_ids = None
    return llm, prompt, stop_token_ids

```

```

# Whisper
def run_whisper(question: str, audio_count: int):
    assert audio_count == 1, (
        "Whisper only support single audio input per prompt")
    model_name = "openai/whisper-large-v3-turbo"

    prompt = "<|startoftranscript|>"

    llm = LLM(model=model_name,
               max_model_len=448,
               max_num_seqs=5,
               limit_mm_per_prompt={
                   "audio": audio_count})
    stop_token_ids = None
    return llm, prompt, stop_token_ids

```

```

model_example_map = {

    "minicpmo": run_minicpmo,
    "phi4_mm": run_phi4mm,
    "qwen2_audio": run_qwen2_audio

```

```

        "qwen2_audio": run_qwen2_audio,
        "ultravox": run_ultravox,
        "whisper": run_whisper,
    }

def main(args):
    model = args.model_type
    if model not in model_example_map:
        raise ValueError(f"Model type {
            model} is not supported.")

    audio_count = args.num_audios
    llm, prompt, stop_token_ids = model_example_map[model](
        question_per_audio_count[audio_count], audio_count)

    # We set temperature to 0.2 so that outputs can be different
    # even when all prompts are identical when running batch inference.
    sampling_params = SamplingParams(temperature=0.2,
                                      max_tokens=64,
                                      stop_token_ids=stop_token_ids)

    mm_data = {
    }

    if audio_count > 0:
        mm_data = {

            "audio": [
                asset.audio_and_sample_rate
                for asset in audio_assets[:audio_count]
            ]

        }

    assert args.num_prompts > 0
    inputs = {
        "prompt": prompt, "multi_modal_data": mm_data}

    if args.num_prompts > 1:
        # Batch inference
        inputs = [inputs] * args.num_prompts

    outputs = llm.generate(inputs, sampling_params=sampling_params)

    for o in outputs:
        generated_text = o.outputs[0].text
        print(generated_text)

if __name__ == "__main__":

```



```

parser = FlexibleArgumentParser(
    description='Demo on using vLLM for offline inference with '
    'audio language models')
parser.add_argument('--model-type',
                    '-m',
                    type=str,
                    default="ultravox",
                    choices=model_example_map.keys(),
                    help='Huggingface "model_type".')
parser.add_argument('--num-prompts',
                    type=int,
                    default=1,
                    help='Number of prompts to run.')
parser.add_argument("--num-audios",
                    type=int,
                    default=1,
                    choices=[0, 1, 2],
                    help="Number of audio items per prompt.")

args = parser.parse_args()
main(args)

```

1.4 嵌入输入

要将预计算的嵌入（属于某种数据类型，如图像、视频或音频）直接输入到语言模型中，可以将形状为 `(num_items, feature_size, hidden_size of LM)` 的张量传递给多模态字典中的相应字段。

```

# Inference with image embeddings as input
llm = LLM(model="llava-hf/llava-1.5-7b-hf")

# Refer to the HuggingFace repo for the correct format to use
prompt = "USER: <image>\nWhat is the content of this image?\nASSISTANT:"

# Embeddings for single image
# torch.Tensor of shape (1, image_feature_size, hidden_size of LM)
image_embeds = torch.load(...)

outputs = llm.generate({
    "prompt": prompt,
    "multi_modal_data": {
        "image": image_embeds},
})

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)

```

对于 **Qwen2-VL** 和 **MiniCPM-V** 模型，我们支持在嵌入之外传递额外的参数：

```
# Construct the prompt based on your model
prompt = ...

# Embeddings for multiple images
# torch.Tensor of shape (num_images, image_feature_size, hidden_size of LM)
image_embeds = torch.load(...)

# Qwen2-VL
llm = LLM("Qwen/Qwen2-VL-2B-Instruct", limit_mm_per_prompt={
    "image": 4})
mm_data = {

    "image": {

        "image_embeds": image_embeds,
        # image_grid_thw is needed to calculate positional encoding.
        "image_grid_thw": torch.load(...), # torch.Tensor of shape (1, 3),
    }
}

# MiniCPM-V
llm = LLM("openbmb/MiniCPM-V-2_6", trust_remote_code=True, limit_mm_per_prompt={
    "image": 4})
mm_data = {

    "image": {

        "image_embeds": image_embeds,
        # image_sizes is needed to calculate details of the sliced image.
        "image_sizes": [image.size for image in images], # List of image sizes
    }
}

outputs = llm.generate({

    "prompt": prompt,
    "multi_modal_data": mm_data,
})

for o in outputs:
    generated_text = o.outputs[0].text
    print(generated_text)
```

2. 在线服务

我们的 OpenAI 兼容服务器通过 **Chat Completions API** 支持多模态数据。

重要提示

使用 Chat Completions API 需要一个聊天模板。

虽然大多数模型都自带聊天模板，但对于其他模型，你需要自己定义一个聊天模板。聊天模板可以根据模型 HuggingFace 仓库的文档推断出来。例如，LLaVA-1.5 ([llava-hf/llava-1.5-7b-hf](#)) 需要一个聊天模板，你可以在以下代码中找到示例。

```
template_llava.jinja
```

```
{
    %- if messages[0]['role'] == 'system' -%}
{
    %- set system_message = messages[0]['content'] -%}
{
    %- set messages = messages[1:] -%}
{
    %- else -%}
{
    % set system_message = '' -%}
{
    %- endif -%}

    {
        bos_token + system_message }}
{
    %- for message in messages -%}
{
    %- if (message['role'] == 'user') != (loop.index0 % 2 == 0) -%}
    {
        raise_exception('Conversation roles must alternate user/assistant/user/assis
tant/...') }}
    {
        %- endif -%}

{
    %- if message['role'] == 'user' -%}
    {
        {
            'USER: ' + message['content'] + '\n' }}
        {
            %- elif message['role'] == 'assistant' -%}
            {
                {
```

```

        'ASSISTANT: ' + message['content'] + eos_token + '\n' }}

{
    %- endif -%}

{
    %- endfor -%}

{
    %- if add_generation_prompt -%}
    {
        'ASSISTANT:' }}
{
    % endif %}

```

2.1 图像输入

图像输入支持遵循 **OpenAI Vision API** 的规范。以下是一个使用 **Phi-3.5-Vision** 的简单示例。

第一步：启动 OpenAI 兼容服务器

运行以下命令启动服务器：

```

vllm serve microsoft/Phi-3.5-vision-instruct --task generate \
    --trust-remote-code --max-model-len 4096 --limit-mm-per-prompt image=2

```

第二步：使用 OpenAI 客户端

你可以按照以下方式使用 OpenAI 客户端：

```

from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

# Single-image input inference
image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"

chat_response = client.chat.completions.create(
    model="microsoft/Phi-3.5-vision-instruct",
    messages=[{

        "role": "user",
        "content": [

```

```

        # NOTE: The prompt formatting with the image token `<image>` is not needed
        # since the prompt will be processed automatically by the API server.
        {
            "type": "text", "text": "What's in this image?"},
        {
            "type": "image_url", "image_url": {
                "url": image_url}},
    ],
    }],
)
print("Chat completion output:", chat_response.choices[0].message.content)

# Multi-image input inference
image_url_duck = "https://upload.wikimedia.org/wikipedia/commons/d/da/2015_Kaczka_krzy%C5%BCowka_w_wodzie_%28samiec%29.jpg"
image_url_lion = "https://upload.wikimedia.org/wikipedia/commons/7/77/002_The_lion_king_Snyggve_in_the_Serengeti_National_Park_Photo_by_Giles_Laurent.jpg"

chat_response = client.chat.completions.create(
    model="microsoft/Phi-3.5-vision-instruct",
    messages=[{
        "role": "user",
        "content": [
            {
                "type": "text", "text": "What are the animals in these images?"},
            {
                "type": "image_url", "image_url": {
                    "url": image_url_duck}},
            {
                "type": "image_url", "image_url": {
                    "url": image_url_lion}},
        ],
    }],
)
print("Chat completion output:", chat_response.choices[0].message.content)

```

完整实例如下: `openai_chat_completion_client_for_multimodal.py`

```

# SPDX-License-Identifier: Apache-2.0
"""An example showing how to use vLLM to serve multimodal models
and run online serving with OpenAI client.

Launch the vLLM server with the following command:

(single image inference with Llava)
vllm serve llava-hf/llava-1.5-7b-hf --chat-template template_llava.jinja

```

```

(multi-image inference with Phi-3.5-vision-instruct)
vllm serve microsoft/Phi-3.5-vision-instruct --task generate \
    --trust-remote-code --max-model-len 4096 --limit-mm-per-prompt image=2

(audio inference with Ultravox)
vllm serve fixie-ai/ultravox-v0_5-llama-3_2-1b --max-model-len 4096
"""
import base64

import requests
from openai import OpenAI

from vllm.utils import FlexibleArgumentParser

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

models = client.models.list()
model = models.data[0].id

def encode_base64_content_from_url(content_url: str) -> str:
    """Encode a content retrieved from a remote url to base64 format."""

    with requests.get(content_url) as response:
        response.raise_for_status()
        result = base64.b64encode(response.content).decode('utf-8')

    return result

# Text-only inference
def run_text_only() -> None:
    chat_completion = client.chat.completions.create(
        messages=[{

            "role": "user",
            "content": "What's the capital of France?"
        }],
        model=model,

```

```

        max_completion_tokens=64,
    )

    result = chat_completion.choices[0].message.content
    print("Chat completion output:", result)

# Single-image input inference
def run_single_image() -> None:

    ## Use image url in the payload
    image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"
    chat_completion_from_url = client.chat.completions.create(
        messages=[{

            "role":
            "user",
            "content": [
                {

                    "type": "text",
                    "text": "What's in this image?"
                },
                {

                    "type": "image_url",
                    "image_url": {

                        "url": image_url
                    },
                },
            ],
            model=model,
            max_completion_tokens=64,
        )

    result = chat_completion_from_url.choices[0].message.content
    print("Chat completion output from image url:", result)

    ## Use base64 encoded image in the payload
    image_base64 = encode_base64_content_from_url(image_url)
    chat_completion_from_base64 = client.chat.completions.create(
        messages=[{

            "role":
            "user",

```

```

        "user",
        "content": [
            {
                "type": "text",
                "text": "What's in this image?"
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{
image_base64}"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from base64 encoded image:", result)

# Multi-image input inference
def run_multi_image() -> None:
    image_url_duck = "https://upload.wikimedia.org/wikipedia/commons/d/da/2015_Kaczka_krzakowa.jpg"
    image_url_lion = "https://upload.wikimedia.org/wikipedia/commons/7/77/002_The_lion_king.jpg"
    chat_completion_from_url = client.chat.completions.create(
        messages=[{
            "role": "user",
            "content": [
                {
                    "type": "text",
                    "text": "What are the animals in these images?"
                },
                {
                    "type": "image_url",
                    "image_url": {

```



```

        "url": image_url_duck
    },
    },
    {

        "type": "image_url",
        "image_url": {

            "url": image_url_lion
        },
    },
],
}],
model=model,
max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output:", result)

# Video input inference
def run_video() -> None:
    video_url = "http://commondatastorage.googleapis.com/gtv-videos-bucket/sample/ForBigg
erFun.mp4"
    video_base64 = encode_base64_content_from_url(video_url)

    ## Use video url in the payload
    chat_completion_from_url = client.chat.completions.create(
        messages=[{

            "role":
            "user",
            "content": [
                {

                    "type": "text",
                    "text": "What's in this video?"
                },
                {

                    "type": "video_url",
                    "video_url": {

                        "url": video_url
                    },
                },
            ],
        },
    ],
    },

```

```

    ],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from image url:", result)

## Use base64 encoded video in the payload
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this video?"
            },
            {

                "type": "video_url",
                "video_url": {

                    "url": f"data:video/mp4;base64,{
                    video_base64}"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from base64 encoded image:", result)

# Audio input inference
def run_audio() -> None:
    from vllm.assets.audio import AudioAsset

    audio_url = AudioAsset("winning_call").url
    audio_base64 = encode_base64_content_from_url(audio_url)

# OpenAI-compatible schema (input audio)

```

```

chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this audio?"
            },
            {

                "type": "input_audio",
                "input_audio": {

                    # Any format supported by Librosa is supported
                    "data": audio_base64,
                    "format": "wav"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from input audio:", result)

# HTTP URL
chat_completion_from_url = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this audio?"
            },
            {

                "type": "audio_url",
                "audio_url": {

```

```

        # Any format supported by librosa is supported
        "url": audio_url
    },
    },
],
}],
model=model,
max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from audio url:", result)

# base64 URL
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this audio?"
            },
            {

                "type": "audio_url",
                "audio_url": {

                    # Any format supported by librosa is supported
                    "url": f"data:audio/ogg;base64,{
audio_base64}"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from base64 encoded audio:", result)

```

```
example_function_map = {
```

```
    "text_only": run_text_only
```

```

    "text-only": run_text_only,
    "single-image": run_single_image,
    "multi-image": run_multi_image,
    "video": run_video,
    "audio": run_audio,
}

def main(args) -> None:
    chat_type = args.chat_type
    example_function_map[chat_type]()

if __name__ == "__main__":
    parser = FlexibleArgumentParser(
        description='Demo on using OpenAI client for online serving with '
        'multimodal language models served with vLLM.')
    parser.add_argument('--chat-type',
                        '-c',
                        type=str,
                        default="single-image",
                        choices=list(example_function_map.keys()),
                        help='Conversation type with multimodal data.')
    args = parser.parse_args()
    main(args)

```

小贴士

vLLM 支持从本地文件路径加载数据：你可以在启动 API 服务器/引擎时通过 `--allowed-local-media-path` 指定允许的本地媒体路径，并在 API 请求中将文件路径作为 `url` 传递。

小贴士

无需在 API 请求的文本内容中放置图像占位符——图像内容已经通过专门的字段表示。事实上，你可以通过在文本中交替插入文本和图像内容来实现图像占位符的效果。

** 注意 **

默认情况下，通过 HTTP URL 获取图像的超时时间为 5 秒。你可以通过设置以下环境变量来覆盖此默认值：

```
export VLLM_IMAGE_FETCH_TIMEOUT=<timeout>
```

2.2 视频输入

你可以通过 `video_url` 传递视频文件，而不是 `image_url`。以下是一个使用 **LLaVA-OneVision** 的简单示例。

第一步：启动 **OpenAI** 兼容服务器

运行以下命令启动服务器：

```
vllm serve llava-hf/llava-onevision-qwen2-0.5b-ov-hf --task generate --max-model-len 8192
```

第二步：使用 **OpenAI** 客户端

你可以按照以下方式使用 OpenAI 客户端：

```

from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

video_url = "http://commondatastorage.googleapis.com/gtv-videos-bucket/sample/ForBiggerFun.mp4"

## Use video url in the payload
chat_completion_from_url = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this video?"
            },
            {

                "type": "video_url",
                "video_url": {

                    "url": video_url
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from image url:", result)

```

完整实例如下: `openai_chat_completion_client_for_multimodal.py`

```

# SPDX-License-Identifier: Apache-2.0
"""An example showing how to use vLLM to serve multimodal models
and run online serving with OpenAI client.

```

Launch the vLLM server with the following command:

(single image inference with Llava)

```
vllm serve llava-hf/llava-1.5-7b-hf --chat-template template_llava.jinja
```

(multi-image inference with Phi-3.5-vision-instruct)

```
vllm serve microsoft/Phi-3.5-vision-instruct --task generate \
    --trust-remote-code --max-model-len 4096 --limit-mm-per-prompt image=2
```

(audio inference with Ultravox)

```
vllm serve fixie-ai/ultravox-v0_5-llama-3_2-1b --max-model-len 4096
```

```
"""
```

```
import base64
```

```
import requests
```

```
from openai import OpenAI
```

```
from vllm.utils import FlexibleArgumentParser
```

```
# Modify OpenAI's API key and API base to use vLLM's API server.
```

```
openai_api_key = "EMPTY"
```

```
openai_api_base = "http://localhost:8000/v1"
```

```
client = OpenAI(
```

```
    # defaults to os.environ.get("OPENAI_API_KEY")
```

```
    api_key=openai_api_key,
```

```
    base_url=openai_api_base,
```

```
)
```

```
models = client.models.list()
```

```
model = models.data[0].id
```

```
def encode_base64_content_from_url(content_url: str) -> str:
```

```
    """Encode a content retrieved from a remote url to base64 format."""
```

```
    with requests.get(content_url) as response:
```

```
        response.raise_for_status()
```

```
        result = base64.b64encode(response.content).decode('utf-8')
```

```
    return result
```

```
# Text-only inference
```

```
def run_text_only() -> None:
```

```
    chat_completion = client.chat.completions.create(
```

```
        messages=[{
```



```

        "role": "user",
        "content": "What's the capital of France?"
    ]],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion.choices[0].message.content
print("Chat completion output:", result)

# Single-image input inference
def run_single_image() -> None:

    ## Use image url in the payload
    image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-nature-boardwalk.jpg/2560px-Gfp-wisconsin-madison-the-nature-boardwalk.jpg"
    chat_completion_from_url = client.chat.completions.create(
        messages=[{

            "role":
            "user",
            "content": [
                {

                    "type": "text",
                    "text": "What's in this image?"
                },
                {

                    "type": "image_url",
                    "image_url": {

                        "url": image_url
                    },
                },
            ],
        }],
        model=model,
        max_completion_tokens=64,
    )

    result = chat_completion_from_url.choices[0].message.content
    print("Chat completion output from image url:", result)

    ## Use base64 encoded image in the payload

```

```

image_base64 = encode_base64_content_from_url(image_url)
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [

            {

                "type": "text",
                "text": "What's in this image?"

            },
            {

                "type": "image_url",
                "image_url": {

                    "url": f"data:image/jpeg;base64,{
image_base64}"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from base64 encoded image:", result)

# Multi-image input inference
def run_multi_image() -> None:
    image_url_duck = "https://upload.wikimedia.org/wikipedia/commons/d/da/2015_Kaczka_krzy%C5%BCowka_w_wodzie_%28samiec%29.jpg"
    image_url_lion = "https://upload.wikimedia.org/wikipedia/commons/7/77/002_The_lion_killing_Snyggve_in_the_Serengeti_National_Park_Photo_by_Giles_Laurent.jpg"
    chat_completion_from_url = client.chat.completions.create(
        messages=[{

            "role":
            "user",
            "content": [

                {

                    "type": "text",
                    "text": "What are the animals in these images?"

                },
            ],
        }],
        model=model,
        max_completion_tokens=64,
    )

    result = chat_completion_from_url.choices[0].message.content
    print("Chat completion output from multiple image URLs:", result)

```

```

    },
    {
        "type": "image_url",
        "image_url": {
            "url": image_url_duck
        },
    },
    {
        "type": "image_url",
        "image_url": {
            "url": image_url_lion
        },
    },
],
}],
model=model,
max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output:", result)

# Video input inference
def run_video() -> None:
    video_url = "http://commondatastorage.googleapis.com/gtv-videos-bucket/sample/ForBigg
erFun.mp4"
    video_base64 = encode_base64_content_from_url(video_url)

    ## Use video url in the payload
    chat_completion_from_url = client.chat.completions.create(
        messages=[{
            "role":
            "user",
            "content": [
                {
                    "type": "text",
                    "text": "What's in this video?"
                },
                {
                    "type": "video_url",

```

```

        "video_url": {
            "url": video_url
        },
    },
],
}],
model=model,
max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from image url:", result)

## Use base64 encoded video in the payload
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{
        "role":
        "user",
        "content": [
            {
                "type": "text",
                "text": "What's in this video?"
            },
            {
                "type": "video_url",
                "video_url": {
                    "url": f"data:video/mp4;base64,{
video_base64}"
                },
            },
        ],
    },
    ],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from base64 encoded image:", result)

# Audio input inference
def run_audio() -> None:
    from vllm.assets.audio import AudioAsset

```

```

from vllm.assets.audio import AudioAsset

audio_url = AudioAsset("winning_call").url
audio_base64 = encode_base64_content_from_url(audio_url)

# OpenAI-compatible schema (`input_audio`)
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [

            {

                "type": "text",
                "text": "What's in this audio?"

            },
            {

                "type": "input_audio",
                "input_audio": {

                    # Any format supported by librosa is supported
                    "data": audio_base64,
                    "format": "wav"

                },

            },

        ],

    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from input audio:", result)

# HTTP URL
chat_completion_from_url = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [

            {

                "type": "text",
                "text": "What's in this audio?"

            },

```

```

        {
            "type": "audio_url",
            "audio_url": {

                # Any format supported by librosa is supported
                "url": audio_url
            },
        },
    ],
}],
model=model,
max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from audio url:", result)

# base64 URL
chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role":
        "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this audio?"
            },
            {

                "type": "audio_url",
                "audio_url": {

                    # Any format supported by librosa is supported
                    "url": f"data:audio/ogg;base64,{
audio_base64}"
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content

```

```

print("Chat completion output from base64 encoded audio:", result)

example_function_map = {

    "text-only": run_text_only,
    "single-image": run_single_image,
    "multi-image": run_multi_image,
    "video": run_video,
    "audio": run_audio,
}

def main(args) -> None:
    chat_type = args.chat_type
    example_function_map[chat_type]()

if __name__ == "__main__":
    parser = FlexibleArgumentParser(
        description='Demo on using OpenAI client for online serving with '
        'multimodal language models served with vLLM.')
    parser.add_argument('--chat-type',
                        '-c',
                        type=str,
                        default="single-image",
                        choices=list(example_function_map.keys()),
                        help='Conversation type with multimodal data.')
    args = parser.parse_args()
    main(args)

```

注意

默认情况下，通过 HTTP URL 获取视频的超时时间为 30 秒。你可以通过设置以下环境变量来覆盖此默认值：

```
export VLLM_VIDEO_FETCH_TIMEOUT=<timeout>
```

2.3 音频输入

音频输入支持遵循 **OpenAI Audio API** 的规范。以下是一个使用 **Ultravox-v0.5-1B** 的简单示例。

第一步：启动 OpenAI 兼容服务器

运行以下命令启动服务器：

```
vllm serve fixie-ai/ultravox-v0_5-llama-3_2-1b
```

第二步：使用 OpenAI 客户端

你可以按照以下方式使用 OpenAI 客户端：

```
import base64
import requests
from openai import OpenAI
from vllm.assets.audio import AudioAsset

def encode_base64_content_from_url(content_url: str) -> str:
    """Encode a content retrieved from a remote url to base64 format."""

    with requests.get(content_url) as response:
        response.raise_for_status()
        result = base64.b64encode(response.content).decode('utf-8')

    return result

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

# Any format supported by Librosa is supported
audio_url = AudioAsset("winning_call").url
audio_base64 = encode_base64_content_from_url(audio_url)

chat_completion_from_base64 = client.chat.completions.create(
    messages=[{

        "role": "user",
        "content": [
            {

                "type": "text",
                "text": "What's in this audio?"
            },
            {

                "type": "input_audio",
                "input_audio": {

                    "data": audio_base64,
                    "format": "wav"
                }
            },
        ],
    }],
)
```



```

    },
    ]],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_base64.choices[0].message.content
print("Chat completion output from input audio:", result)

```

或者你可以传递 `audio_url`，这是音频输入中与图像输入中的 `image_url` 对应的字段：

```

chat_completion_from_url = client.chat.completions.create(
    messages=[{
        "role": "user",
        "content": [
            {
                "type": "text",
                "text": "What's in this audio?"
            },
            {
                "type": "audio_url",
                "audio_url": {
                    "url": audio_url
                },
            },
        ],
    }],
    model=model,
    max_completion_tokens=64,
)

result = chat_completion_from_url.choices[0].message.content
print("Chat completion output from audio url:", result)

```

完整示例如：`openai_chat_completion_client_for_multimodal.py`。

注意

默认情况下，通过 HTTP URL 获取音频的超时时间为 10 秒。你可以通过设置以下环境变量来覆盖此默认值：

```
export VLLM_AUDIO_FETCH_TIMEOUT=<timeout>
```

2.4 嵌入输入

要将预计算的嵌入（属于某种数据类型，如图像、视频或音频）直接输入到语言模型中，可以将形状为 `(num_items, feature_size, hidden_size of LM)` 的张量传递给多模态字典中的相应字段。

2.4.1 图像嵌入输入

对于图像嵌入，你可以将 base64 编码的张量传递给 `image_embeds` 字段。以下示例展示了如何将图像嵌入传递给 OpenAI 服务器：

```
image_embedding = torch.load(...)
grid_thw = torch.load(...) # Required by Qwen/Qwen2-VL-2B-Instruct

buffer = io.BytesIO()
torch.save(image_embedding, buffer)
buffer.seek(0)
binary_data = buffer.read()
base64_image_embedding = base64.b64encode(binary_data).decode('utf-8')

client = OpenAI(
    # defaults to os.environ.get("OPENAI_API_KEY")
    api_key=openai_api_key,
    base_url=openai_api_base,
)

# Basic usage - this is equivalent to the LLaVA example for offline inference
model = "llava-hf/llava-1.5-7b-hf"
embeds = {

    "type": "image_embeds",
    "image_embeds": f"{
        base64_image_embedding}"
}

# Pass additional parameters (available to Qwen2-VL and MiniCPM-V)
model = "Qwen/Qwen2-VL-2B-Instruct"
embeds = {

    "type": "image_embeds",
    "image_embeds": {

        "image_embeds": f"{
            base64_image_embedding}" , # Required
        "image_grid_thw": f"{
            base64_image_grid_thw}" # Required by Qwen/Qwen2-VL-2B-Instruct
    },
}

model = "openbmb/MiniCPM-V-2_6"
```

```

embeds = {

    "type": "image_embeds",
    "image_embeds": {

        "image_embeds": f"{
            base64_image_embedding}" , # Required
        "image_sizes": f"{
            base64_image_sizes}" # Required by openbmb/MiniCPM-V-2_6
    },
}

chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user", "content": [
                {
                    "type": "text",
                    "text": "What's in this image?",
                },
                embeds,
            ],
        },
    ],
    model=model,
)

```

注意

- 只有一个消息可以包含 `{"type": "image_embeds"}`。 - 如果与需要额外参数的模型一起使用，你还必须为每个参数提供一个张量，例如 `image_grid_thw`、`image_sizes` 等。